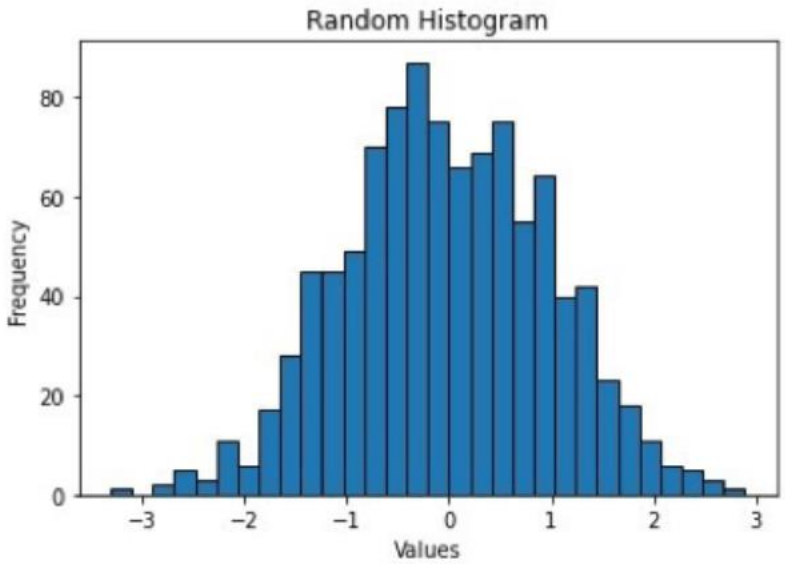
Module 4

Statistical Thinking

Distributions – Representing and plotting histograms

Histogram data is a fundamental tool in visualization, which provides a simple yet powerful way to understand the distribution of data. Whether you are new to data analysis or looking to sharpen your skills, histogram is an important tool for summarizing and imagining the data points.

A type of histogram is a type of graphical representation that refers to the distribution of numerical values. It consists of a set of vertical bars, where each time represents a range of values, and the height of the bar indicates the frequency or count of data points falling within that range.



Histograms are usually used to imagine the size of the data set in data and data analysis and identify the pattern, such as the presence of outlair or obliqueness. They are also useful to compare the distribution of various data sets or to identify trends over time.

The picture above shows how 1000 random data points from a general distribution and standard deviation of 1 are plotted into a histogram with 30 coaches and black edges.

We can create a histogram using Matplotlib by following a series of steps. Following the import details of libraries, the code generates a set of 1000 random data points from a general distribution with 0 and standard deviations, using 0 and standard deviations.

Python has a powerful tool for making plt.hist () function histogram. By providing data, number of compartments, bar color, and edge color as input, this function produces a histogram plot.

To enhance visualization, Xlabel (), ylabel (), and title () functions are used to add the labeled to X and Y axes, as well as a title for the plot.

Finally, the show () function is employed to display histograms on the screen, which allows for detailed analysis and interpretation.

```
import matplotlib.pyplot as plt
import numpy as np

# generate data
data = np.random.normal(0, 1, 1000)

# create histogram
plt.hist(data, bins=30, color='blue',
edgecolor='black')

# set labels and title
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.title('Histogram of Data')

# display plot
plt.show()
```

Outliers

Outlier are extreme values that differ from most other data points in a dataset. They can have a major impact on your statistical analysis and slant the results of any hypothesis tests. It is important to carefully identify the potential outliers in your dataset and to treat them properly for accurate results. There are four ways to identify outliers: Sorting method Data visualization system Statistical test (jade score) Contradictory limit system The outlier has values at the extreme ends of a dataset.

Descriptive figures often involve using some numbers abbreviated to distribution. An important aspect of a distribution is where its center is located. Central trend measures are first discussed. Another aspect of a distribution is how it spreads. In other words, how different the number in distribution is from each other. The second section describes measures of variability. Distribution can vary in size. Some distribution are symmetrical while others have long tails in just one direction. The third section describes measures of distribution size. Concern of the last two volumes (1) how changes affect the measures summarizing distribution and (2) variance sum law, a significant relationship that includes a measure of variability.

🔹 1. Central Tendency (Where the data is centered)

Measure	Description	Function (Python - Pandas)
Mean	Average value	`df['col'].mean()`
Median	Middle value when data is sorted	`df['col'].median()`
Mode	Most frequent value	`df['col'].mode()`

🔹 2. Dispersion (How spread out the data is)

Measure	Description	Function (Pandas)
Range	Difference between max and min	`df['col'].max() - df['col'].min()`
Variance	Average of squared deviations from the mean	`df['col'].var()`
Standard Deviation	Spread around the mean (square root of variance)	`df['col'].std()`
IQR (Interquartile Range)	Middle 50% of data (Q3 - Q1)	`Q3 - Q1`

🔹 3. Shape of Distribution

- **Skewness**: Measures asymmetry of data
  - Positive skew → tail on right
  - Negative skew → tail on left
  - `df['col'].skew()`
- **Kurtosis**: Measures "tailedness" of distribution
  - High kurtosis → heavy tails (outliers)
  - `df['col'].kurt()`

Variance

The variance is a statistical measurement that is used to determine the spread of numbers in a data set in relation to the average value or mean. The standard deviation class will give us variance. Using the variance we can evaluate how or squeezed the distribution.

Statistics can have two types of versions, namely, sample variance and population variance. The symbol of variance is given by σ2. The variance is widely used in hypothesis tests, examining the good of the fit, and monte carlo sampling. To check how individual data points vary in relation to using variance. In this article, we will take a look at the qualities of definition, example, formula, application and variance. Variation is a solution to spread. A remedy for spread is a volume that is used to check the variability of data about the average value. Data can be of two types - grouped and ungreed. When data is expressed as square interval, it is known as grouped data. On the other hand, if the data has individual data points, it is called non -unguided data. Both sample and population variance can be determined for data.

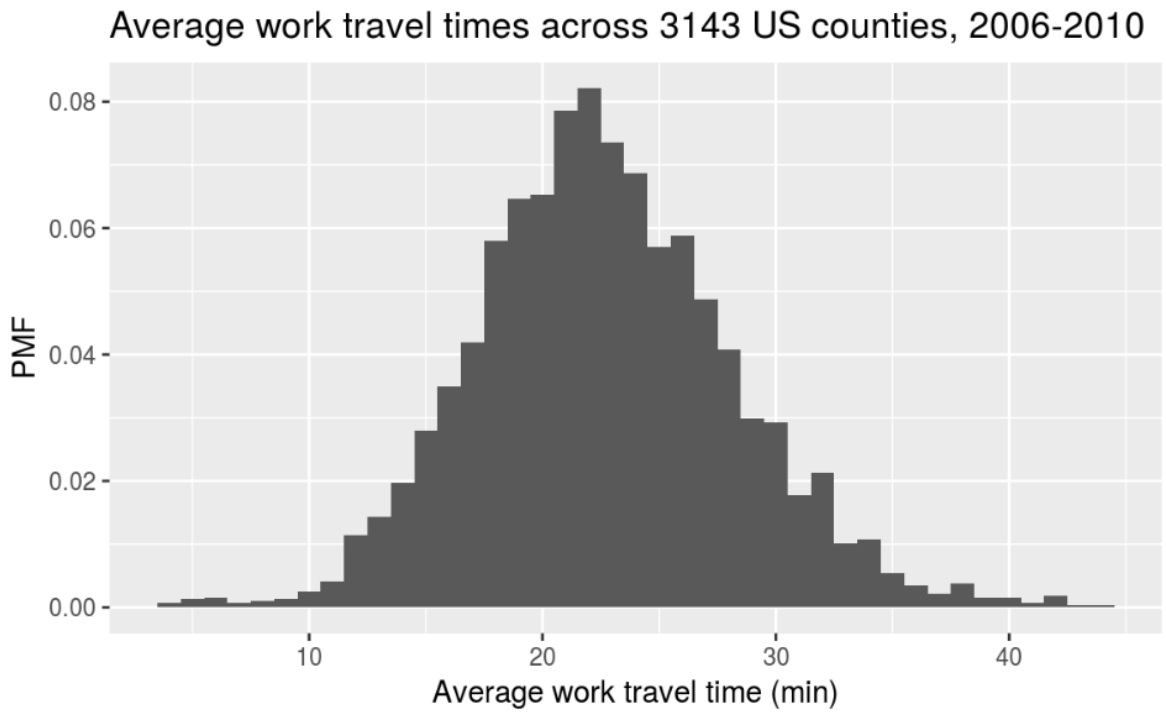
	Population	Sample
Ungrouped	$\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \bar{x})^2}{N}$	$\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \bar{x})^2}{N - 1}$
Grouped	$\sigma^2 = \sum_{i=1}^{N} \frac{f(M_i - \bar{X})^2}{N}$	$\sigma^2 = \sum_{i=1}^{N} \frac{f(M_i - \bar{X})^2}{N - 1}$

Probability mass function – Plotting PMFs

The probability mass function (PMF) represents a distribution by sorting the data into the compartment (a lot of frequency is like histogram) and then combines a possibility with each bin in delivery. A possibility is a frequency expressed as a fraction of sample size n. Therefore we can directly convert a frequency histogram by dividing the count in each bin in PMF by dividing sample size n. This process is called normalization.

The syntax for plotting PMF using ggplot2 is almost the same what you will use to create a frequency histogram. An amendment is that you need to include y = .. density .. inside AES (). As a simple example, let's do the complete distribution of average work time in advance and plot it as PMF:\

```
county_complete %>%
  ggplot(mapping = aes(x = mean_work_travel, y = ..density..)) +
  geom_histogram(binwidth = 1) +
  labs(title = "Average work travel times across 3143 US counties, 2006-2010",
       x = "Average work travel time (min)",
       y = "PMF")
```



Other visualizations

1. Line Graph

The line graph is the favorite law in the enumeration of graphs as well as various different number of business applications due to the fact they present a trend in a total examined manner.

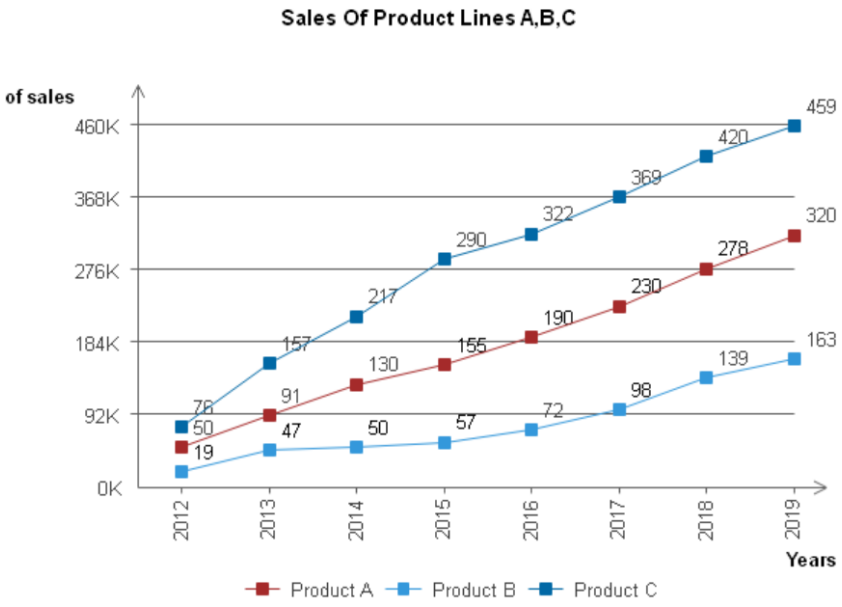
What is line graph?

A line chart (also called a line graph) is a graph that is used to represent the values of something against a given time span.

In the case of your firm, your sales department might place a graph of how the number of sales you have on hand varies with time. The values are shown in forms of data points, connected with straight lines.

When is line graphs use?

You should use it when you need to show trends.You want to present trends of various categories at the same time of a period and to present comparison.



Bar Chart

Bar charts are quite popular types of data displays since you can read them without much effort and draw useful information and they are excellent to compare multiple various types of data.

What is a bar chart?

Bar chart (or bar graph) is a type of a chart used to display data in a form of bars of various height.

The bars are of two types i.e. vertical or horizontal. You do not have to use a particular type.

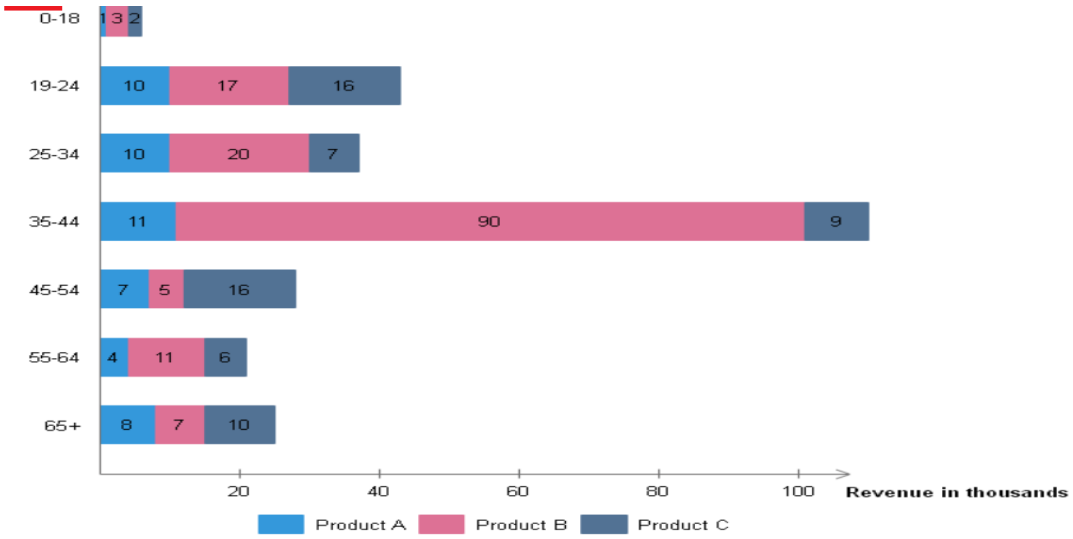
We can easily compare data on each variable at every point of time by the help of bar chart.

As an example, a bar chart might seek to compare the sales of your company in this year to the last year.

What time to use a bar chart?

When it is necessary to compare various categories.

In the event that you have to demonstrate the size of change in data with time.



Column Chart

The column chart is a special kind of bar graph where vertical bars are used to indicate comparison of categories.Whatever can be counted can be represented in column chart.

The column charts are the most appropriate means of depicting the scenario at any given moment (such as the quantity of items sold on a web-shop).

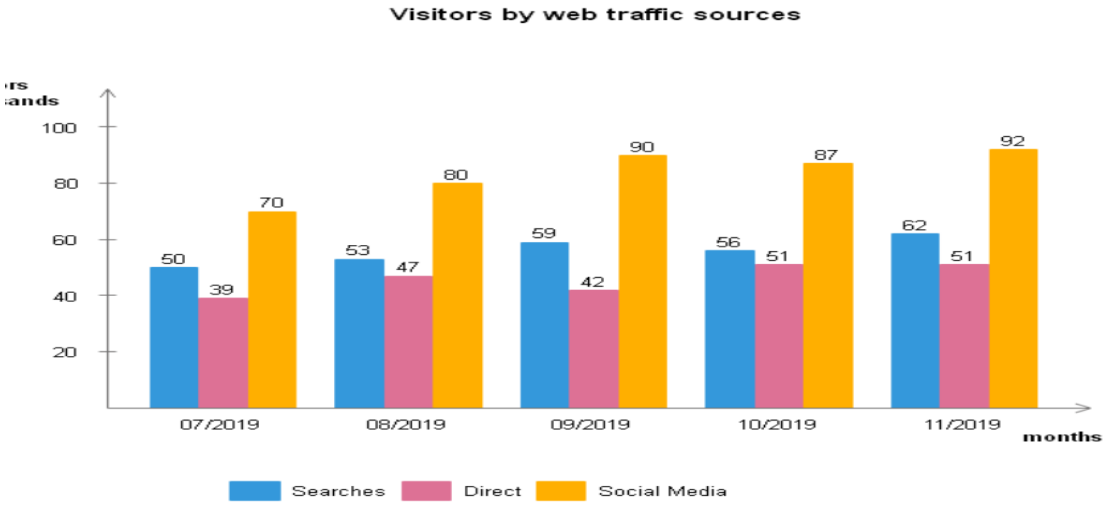
The primary aim of their usage is to focus on the overall numbers as opposed to the trend (trends are better represented by a line chart).

What is the situation to apply a column chart?

You have to demonstrate the comparison side-by-side when you have to compare two different values.

When you would like to underline the distinction between values.

When you wish to point out the cumulative numbers as opposed to the trends.



One of the beautiful types of data visualization is presented in pie charts. On a high level, they are readable easily and they are used in area representational descriptions of relative sizes.

pie chart:

A Pie Chart is a round shaped graph which depicts relativity of sizes of data in terms of pieces of pies.

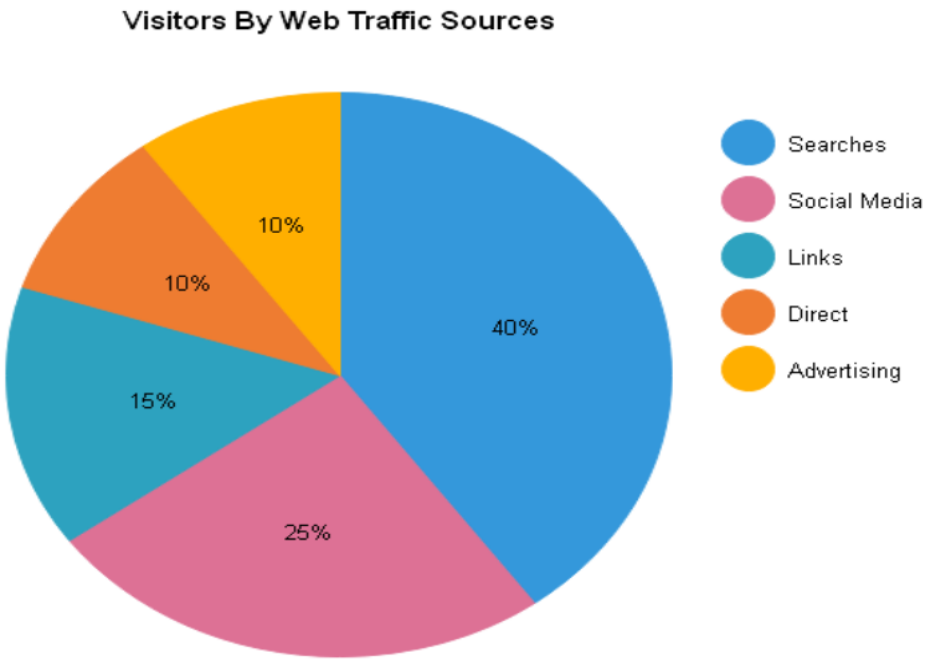
Pie chart is an ideal tool with regard to displaying percentages since it displays each component as a component of the entirety.

The whole pie constitutes 100 percent of a whole. The slices in the pie symbolize parts of the entirety.

When not to use pie chart?

To indicate the percentage that each of the values contributes to the entirety.

When you would like to illustrate the breaking up of a group into smaller parts.



Area Chart:

An area chart is an optimal choice in case you have to represent the data reflecting a time-series relationship.

What is area chart?

An area chart is that kind of a chart that shows the variation of one or more quantities with time. It is like a line graph.

In area charts as well as line graphs, the data values are linked by a line to indicate the value of a quantity at various times. Both of them are good when it comes to displaying trends.

Nevertheless, the area type graphic is not similar to the line graph, as the space between the x-axis and the line is colored. Area charts hence provide a feeling about the total volume.

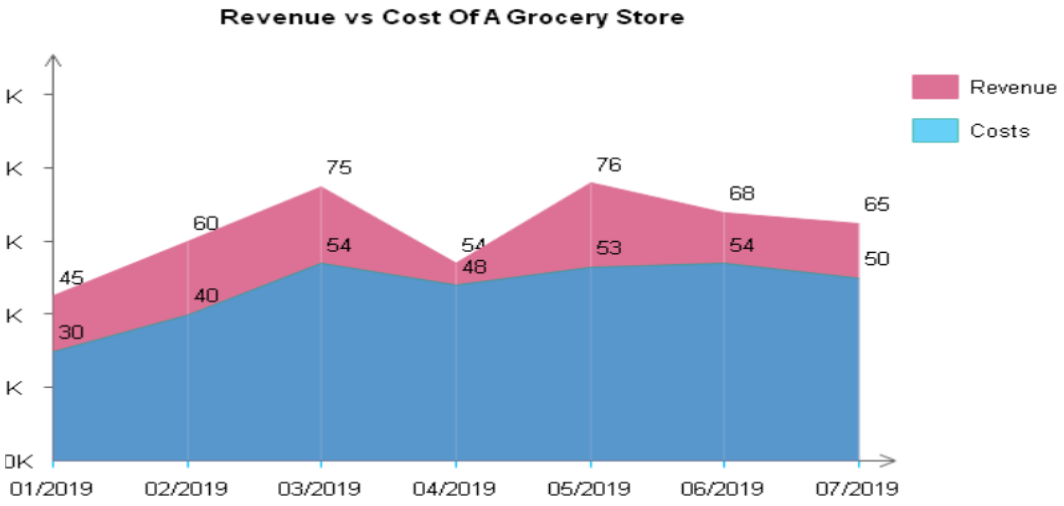
Area charts focus on a trend through time. They do not pay so much attention to a presentation of exact values.

The area charts are also ideal where there is need to exhibit the variation of the different sets of data.

Where to apply an area chart?

In instances where you need to apply a series of lines to draw a comparison between groups (tells aka series).

You need to know the breakdown of the total by groups as well as their whole value when you desire to follow such a value.



Scatter Plot

Scatter plot is also one of the commonly used types of data visualization and it is also referred to as a scatter diagram, scatter graph and correlation chart.Scatter plot is useful in various spheres of the modern world business, biology, social statistics, data science, and etc.

What is Scatter plot?

Scatter plot is a graphical depiction stating a relationship amid two variables. It is aimed at demonstrating the extent of influence that one variable has over another.

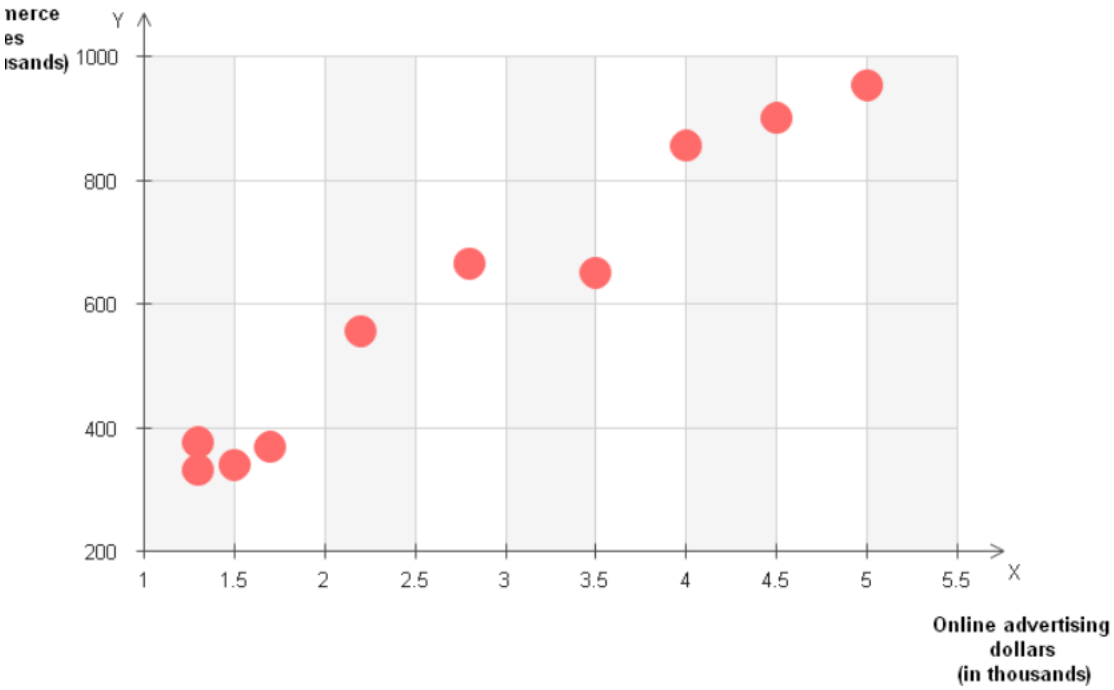
In most cases, in the same formula when there exists an association between 2 variables, the former is referred to as the independent variable. The second variable, as it is known, is called a dependent variable since the values obtainable depend on another variable.

However, there is also a possibility of no kind of relationship between 2 variables.

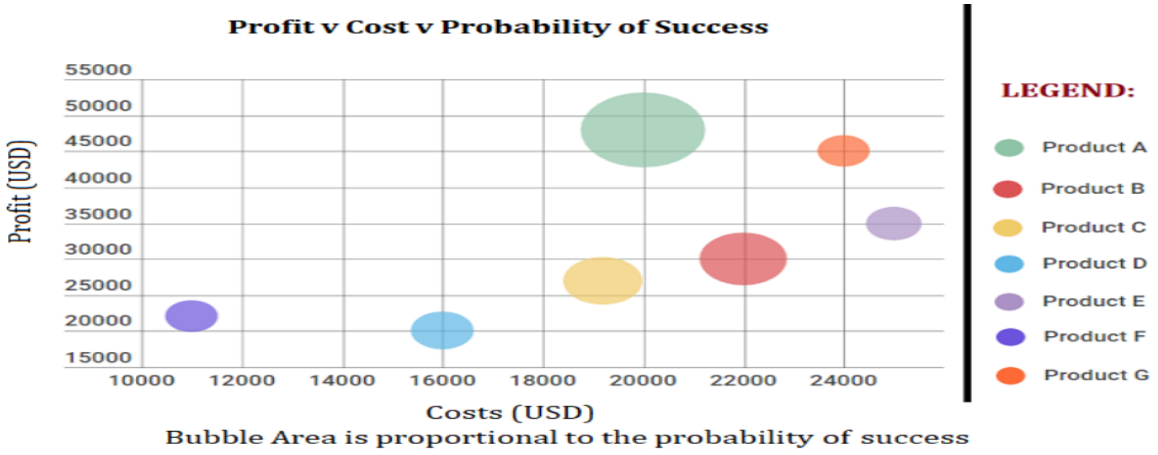
Usage of a Scatter plot.

You ought to use it when you are required to note and demonstrate relationships between two numerical variables.

When one simply wants to graphically represent the relationship between 2 datasets that are large in nature without relating to time.



Bubble Chart:



The class size paradox

Class size paradox Characteristics Paradox A result of statistics Surprise countParadoxClass sizeA statistical anomalyThe class size paradox is an unexpected statistical occurrence

generally employed in the scope of data science and probability to point out bias in averaging and sampling algorithms. It is in close connection with the sampling bias and the paradox of friends.

Definition:
The class size paradox is the fact that the size of the average class experienced by students is bigger than the size of the average calculated on all classes.

Sampling Bias:
In case you sample individuals instead of groups, your estimates will be biased.

Weighted Averages:
Misapplication of averages which do not take into account weights may result in false conclusion.

Survey Design:
In the analysis of survey or user data (e.g. product usage, class feedback) the assumption that the distribution of participants can be skewed is also valid.

Let's say a school has the following classes:

Class	Number of Students
A	10
B	20
C	30

- **Average class size (per class)**:
  $\frac{10+20+30}{3} = 20$ students.
- **Average class size experienced by students (weighted by students)**:

$$\frac{10 \times 10 + 20 \times 20 + 30 \times 30}{10 + 20 + 30} = \frac{100 + 400 + 900}{60} = \frac{1400}{60} \approx 23.3$$

the average class size is 20, the average *student's experience* is closer to 23.3.

Misleading Metrics:
In business intelligence/A/B testing, results may seem skewed when the data has not been normalized correctly.

Data frame indexing

An efficient way to manipulate data is a top priority in data science, and indexing in DataFrames (as in pandas) can speed up and rationalise how we extract, filter and manipulate data.

What is DataFrame?
DataFrame is a 2 dimensional tabular data structure that has:

Columns (labeled with an index),

Columns (marked with headers),

Just like an excel sheet or an SQL table.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Department': ['HR', 'IT', 'Finance']
}

df = pd.DataFrame(data)
```

Types of Indexing

1. Label-Based Indexing (.loc)
Uses row and column labels (names).

Syntax: df.loc[row_label, column_label]

```
df.loc[0, 'Name']      # Output: 'Alice'
df.loc[0]              # Entire first row
df.loc[:, 'Age']       # Entire Age column
```

2.Integer-Based Indexing (.iloc)
Uses row and column positions (like array indexing).
Syntax: df.iloc[row_index, column_index]

```
df.iloc[1, 0]          # Output: 'Bob'
df.iloc[1]             # Entire second row
df.iloc[:, 2]          # Entire third column ('Department')
```

3. Boolean Indexing (Filtering)
Index rows using logical conditions.

```
df[df['Age'] > 28]     # Returns rows where Age > 28
```

4. Set Index
You can also set a column as an index for faster lookup.

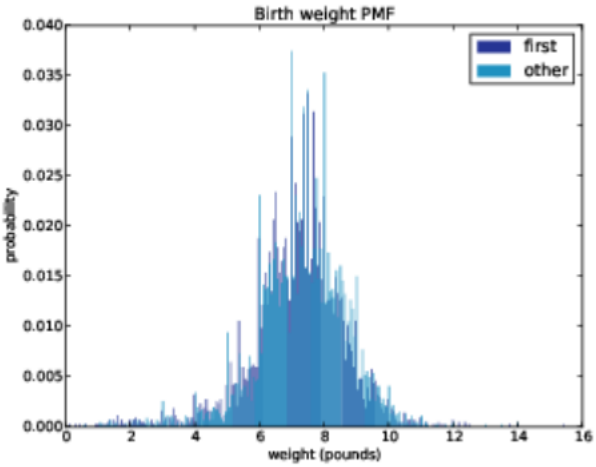
```
df.set_index('Name', inplace=True)
df.loc['Alice']        # Access Alice's row
```

Cumulative distribution functions - Limits of PMFs

Constrains of PMFs

In case the number of values is not large, PMFs will work well. However, the more the values that have to be calculated, the less the probability of each value and the high effect of random noise.

As an example we may be interested in distribution of birth weights. NSFG data Total wgt lb variable stores the weight of a baby at birth, in pounds. The PMF of this quantity of first babies and other is given in Figure below.



PMF of birth weights

In general, such distributions take the shape of a bell of a normal distribution, where the values are higher along the mean and far apart in both ends.

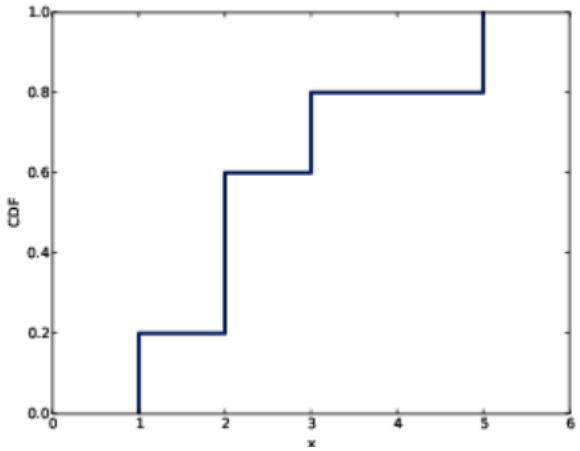
However, components of this figure are not easy to read. Numerous spikes and valleys are present and there are some differences that can be considered visible between the distributions. Which of these features are meaningful, it is hard to tell. Too, it is difficult to observe total trends; e.g., which variance is it that you think has the larger average?

Binning of the data can overcome these problems; i.e., putting data range into non-overlapping (binned) intervals and tabulating the number of values within each bin. Binning may be helpful but bin size is something difficult to correct. By smoothing out the noise they

may also smooth out the useful information, too, should they be large enough.

```
def EvalCdf(sample, x):
    count = 0.0
    for value in sample:
        if value <= x:
            count += 1

    prob = count / len(sample)
    return prob
```



This is essentially equivalent to PercentileRank, but the output stands in the interval [0,1] instead of having the percentile rank in the interval [0,100].

To make an example, consider the sample to be taken as [1, 2, 2, 3, 5]. These are some of its CDF values:

CDF(0) = 0
The CDF (1) = 0.2
The CDF(2) = 0.6
The CDF(3) = 0.8
CDF(4) = 0.8
The likelihood that five goals are scored is 1
We are in a position to assess the CDF of any x and not necessarily the values that are present in the sample. CDF(x) is zero when x is less than minimum in the sample. In the case of x that exceeds the largest value, CDF(x) is 1.

Representing CDFs

Cdf basic procedures offered are:

Prob (x): for a given value x, the probability is calculated as p = CDF (x). The bracket operator is synonymous to Prob.
Value(p): Computes the value, x corresponding to a probability, p; i.e. the inverse CDF of p.

The constructor of Cdf accepts the argument of a list of values, a pandas Series, a Hist, Pmf or another instance of Cdf. The next listing creates a Cdf of the distribution of the length of pregnancy in the NSFG:

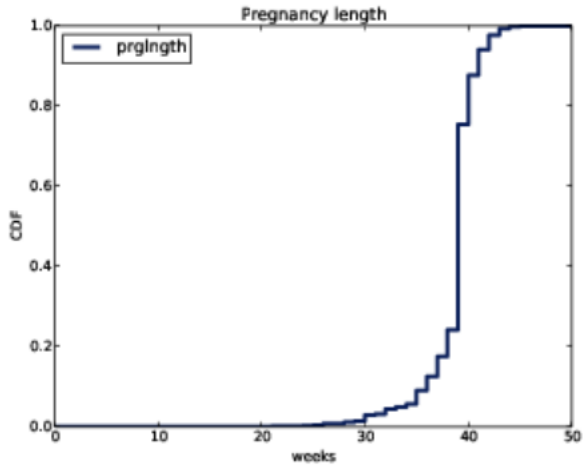


Figure 4.3: CDF of pregnancy length.

```
live, firsts, others = first.MakeFrames()
cdf = thinkstats2.Cdf(live.prglngth, label='prglngth')
```

thinkplot has a function Cdf which graphs Cdfs as lines:

```
thinkplot.Cdf(cdf)
thinkplot.Show(xlabel='weeks', ylabel='CDF')
```

Percentile based statistics

Percentiles refer to the proportion of the scores which lie below a given score. They are a sort of quantile which divides the data into 100 roughly equal sized cell and informs you the location of a score relative to other scores. To illustrate this, somebody who has an IQ of 120 is on 91 st percentile indicating that his or her IQ is greater than 91 other scores.

When you want the relative position of a number, percentiles are a fine thing to have.
Percentiles give you an idea about how a value stands against other values. The simple rule is that where X appears at kth percentile then, X exceeds K percent of the values. Now, let us view how this information may be useful.
. As a result there are various ways of computing percentile. This post lists four procedures. The first three are the practices that analysts resort to in finding the percentiles when examining the actual values of data in moderately sized data sets. The k th percentile is defined differently by the above three definitions as below:

The lowest value that exceeds more than k-per cent of the values.
The least amount that supersedes or equals to k percent of values.
A computed value between two nearest ranks.

Random numbers

The importance of random numbers in data science Researchers, amateurs, and students use random numbers to create such models as random sampling and generate synthetic data and initialize machine learning models.
Random numbers are values to which a number is created so that all the numbers are likely to be chosen equally.

Application	Purpose
Data Splitting	Randomly divide data into training/testing sets
Bootstrapping & Resampling	Create new datasets from samples with replacement
Simulations (e.g., Monte Carlo)	Run repeated random experiments to estimate outcom
Synthetic Data Generation	Create artificial data for testing models
Random Forests & ML Algorithms	Use randomness to increase robustness and reduce overfitting

They are typically pseudo-random in data science explains in data science, the algorithms are typically pseudo-random (i.e. not really random yet sufficiently random to be useful in practice).

Generating Random Numbers:

```
np.random.randint(1, 10, size=5)
# e.g., [7, 2, 4, 9, 1]
```

```
np.random.rand(5)
# e.g., [0.78, 0.12, 0.45, 0.89, 0.33]
```

```
np.random.normal(loc=0, scale=1, size=5)
# e.g., values centered around 0 with standard deviation 1
```

```
data = np.array([1, 2, 3, 4, 5])
np.random.shuffle(data)
```

Comparing percentile ranks:

Percentile ranks are a strong statistical concept in data science and they compare values with reference to a distribution. Two percentiles represent the proportions of observations shown by a dataset that are less than a particular value.

Suppose a student scored at the 90th percentile, or above 90 percent persons and he is better than them.

```python
from scipy import stats
import numpy as np

data = [55, 60, 65, 70, 75, 80, 85, 90, 95, 100]
value = 80

percentile_rank = stats.percentileofscore(data, value)
print(f"{value} is at the {percentile_rank}th percentile.")
# Output: 60.0
```

Comparison of Percentile Ranks:

```python
math_scores = [55, 60, 65, 70, 75, 80, 85]
science_scores = [60, 62, 65, 68, 70, 72, 75]

math_score = 70
science_score = 68

math_percentile = stats.percentileofscore(math_scores, math_score)
science_percentile = stats.percentileofscore(science_scores, science_score)

print(f"Math: {math_percentile}th percentile")
print(f"Science: {science_percentile}th percentile")
```

Modeling distributions - Exponential distribution

Its probability distribution is used when modelling the time between independent events in a Poisson process, in data science and in statistics (i.e., when events occur one at a time at a constant mean rate).

- Exponential distribution is applicable in modelling the time to event.
- It has the assumption that events are independent and occur at random.
- The rate of occurrence = λ (lambda).
- Unlike other continuous distributions it is memoryless.

Feature	Description
Shape	Right-skewed (decays exponentially)
PDF	$f(x;\lambda) = \lambda e^{-\lambda x}$ for $x \geq 0$
Mean	$\frac{1}{\lambda}$
Variance	$\frac{1}{\lambda^2}$
Memoryless Property	The future is independent of the past

Use Case	Example
Queueing systems	Time between customer arrivals at a service counter
Reliability analysis	Time until failure of a machine or component
Web analytics	Time between website hits or API calls
Survival analysis	Lifespan of systems, organisms, etc.
Simulation	Synthetic modeling of real-world events

Modeling Exponential Distribution in Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import expon

# λ = 1.5 (rate parameter)
lambda_val = 1.5
data = np.random.exponential(scale=1/lambda_val, size=1000)

plt.hist(data, bins=30, density=True, alpha=0.6, color='skyblue', label='Histogra

# Plot PDF
x = np.linspace(0, max(data), 1000)
plt.plot(x, expon.pdf(x, scale=1/lambda_val), 'r', label='Exponential PDF')

plt.title('Exponential Distribution (λ=1.5)')
plt.xlabel('Time')
plt.ylabel('Probability Density')
plt.legend()
plt.show()
```

Normal distribution

The Gaussian distribution also referred to as the normal distribution is a bell-shaped probability distribution that is symmetrical about the mean.

It is one of the most significant distributions in statistics and data science since there were many natural and social phenomena that this distribution tends to follow.

Feature	Description
Shape	Bell-shaped, symmetric
PDF (probability density function)	$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
Mean (μ)	Center of the distribution
Standard deviation (σ)	Controls the spread (width of the curve)
Empirical Rule	~68% within 1σ, ~95% within 2σ, ~99.7% within 3σ

Use Case	Example
Error distribution	Residuals in regression often assumed to be normal
Statistical inference	Many tests (t-test, z-test) assume normality
Standardization (Z-scores)	Used to compare different scales
Data normalization	Feature scaling for ML algorithms
Anomaly detection	Values far from the mean are flagged as anomalies

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
  mu = 50      # mean
  sigma = 10   # standard deviation

  # Generate random data
  data = np.random.normal(loc=mu, scale=sigma, size=1000)
```

```
# Plot histogram
plt.hist(data, bins=30, density=True, alpha=0.6, color='skyblue', label='Data')

# Plot the PDF
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
plt.plot(x, norm.pdf(x, mu, sigma), 'r', label='Normal PDF')

plt.title('Normal Distribution (μ=50, σ=10)')
plt.xlabel('Value')
plt.ylabel('Probability Density')
  plt.ylabel('Probability Density')
  plt.legend()
  plt.grid(True)
  plt.show()
```

Lognormal distribution.

P probability distribution -A lognormal distribution is a continuous probability distribution of a random variable, such that the logarithm of the variable follows a normal distribution.

Feature	Description
Shape	Right-skewed, non-negative
Support	$X > 0$ (values are always positive)
PDF	$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} e^{-\frac{(\ln x - \mu)^2}{2\sigma^2}}$
Mean	$e^{\mu+\sigma^2/2}$
Median	e^{μ}
Mode	$e^{\mu-\sigma^2}$

Use Case	Example
Income distribution	Most people earn below average; few earn extremely high
Stock prices	Prices cannot be negative, and changes are multiplicative
Biological measures	Weight, growth rates, or reaction times
Time to failure	When failure rate increases over time

Implementation:

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import lognorm
```

```python
mu = 0       # Mean of log values
sigma = 0.9  # Std dev of log values

# Generate data
data = np.random.lognormal(mean=mu, sigma=sigma, size=1000)

# Plot histogram
plt.hist(data, bins=30, density=True, alpha=0.6, color='skyblue', label='Histogra

# Plot the PDF
x = np.linspace(0.01, max(data), 1000)
pdf = lognorm.pdf(x, s=sigma, scale=np.exp(mu))
plt.plot(x, pdf, 'r', label='Lognormal PDF')

plt.title('Lognormal Distribution (μ=0, σ=0.9)')
plt.xlabel('Value')
plt.ylabel('Probability Density')
plt.legend()
plt.grid(True)
plt.show()
```